

OS-Homework7

计算机 2303 尤颢辰

2026 年 5 月 24 日

1 Question 1:

1.1 Subquestion a

观察图中的情况，四辆车分别从东南西北四个方向进入十字路口，每辆车都占据了一段路面，并阻塞了对面车辆的前进。

1. **互斥条件：**每一个路段在同一时刻只能被一辆车占用，两辆车不能同时占据同一空间位置。
2. **占有并等待：**每辆车都占据着自己当前所在的路段（已获得的资源），同时又在等待前方被其他车占据的路段（请求新资源）才能继续前进。
3. **非抢占条件：**车辆一旦进入某段路面，就不能被强制赶走，即在现实交通规则下不允许倒车，也无法越过其他车辆。资源只能由占有者自愿释放。
4. **循环等待：**形成了一个环形等待链：北向车等待东向车移动，东向车等待南向车移动，南向车等待西向车移动，西向车等待北向车移动。

四个条件同时成立，因此系统处于死锁状态。

1.2 Subquestion b

想要避免死锁，可以应用一个简单的规则：**不要堵住路口。**

即，**车辆只有在确认能够完全通过路口，即对面出口处有足够空间容纳自己的情况下，才允许进入路口区域；否则必须在停止线外等待。**

这条规则破坏了占有并等待条件——车辆在进入路口前必须确保出口畅通，因此它不会出现“已进入路口但前方被堵”的情况，也就不会一边占用路口资源一边等待另一资源。且也可以理解为破坏了循环等待条件，因为永远不会出现四个方向的车同时进入并互相阻塞的环路。

2 Question 2:

2.1 Subquestion a

根据 $\text{Need} = \text{Max} - \text{Allocation}$ ，求出 Need 矩阵：

进程	A	B	C	D
P_0	0	0	0	0
P_1	0	7	5	0
P_2	1	0	0	2
P_3	0	0	2	0
P_4	0	6	4	2

表 1: Need 矩阵

2.2 Subquestion b

执行安全性算法，初始 $Work = Available = (1, 5, 2, 0)$

步骤	进程	Need $\leq$ Work?	执行后 Work
1	P_0	$(0, 0, 0, 0) \leq (1, 5, 2, 0) \checkmark$	$(1, 5, 3, 2)$
2	P_2	$(1, 0, 0, 2) \leq (1, 5, 3, 2) \checkmark$	$(2, 8, 8, 6)$
3	P_1	$(0, 7, 5, 0) \leq (2, 8, 8, 6) \checkmark$	$(3, 8, 8, 6)$
4	P_3	$(0, 0, 2, 0) \leq (3, 8, 8, 6) \checkmark$	$(3, 14, 11, 8)$
5	P_4	$(0, 6, 4, 2) \leq (3, 14, 11, 8) \checkmark$	$(3, 14, 12, 12)$

表 2: 安全性算法执行过程

2.3 Subquestion c

第一步：检查 $Request \leq Need(P_1)$?

由于 $(0, 4, 2, 0) \leq (0, 7, 5, 0)$ ，检查通过。

第二步：检查 $Request \leq Available$?

由于 $(0, 4, 2, 0) \leq (1, 5, 2, 0)$ ，检查通过。

第三步：试探性分配

项目	更新后
Available	$(1, 5, 2, 0) - (0, 4, 2, 0) = (1, 1, 0, 0)$
Allocation(P_1)	$(1, 0, 0, 0) + (0, 4, 2, 0) = (1, 4, 2, 0)$
Need(P_1)	$(0, 7, 5, 0) - (0, 4, 2, 0) = (0, 3, 3, 0)$

表 3: 试探性分配

第四步：再次执行安全性检查， $Work = (1, 1, 0, 0)$

步骤	进程	Need $\leq$ Work?	执行后 Work
1	P_0	$(0, 0, 0, 0) \leq (1, 1, 0, 0) \checkmark$	$(1, 1, 1, 2)$
2	P_2	$(1, 0, 0, 2) \leq (1, 1, 1, 2) \checkmark$	$(2, 4, 6, 6)$
3	P_1	$(0, 3, 3, 0) \leq (2, 4, 6, 6) \checkmark$	$(3, 8, 8, 6)$
4	P_3	$(0, 0, 2, 0) \leq (3, 8, 8, 6) \checkmark$	$(3, 8, 8, 6)$
5	P_4	$(0, 6, 4, 2) \leq (3, 14, 11, 8) \checkmark$	$(3, 14, 12, 12)$

表 4: 分配后的安全性检查

故得出结论，该请求可以立即满足，分配后系统仍处于安全状态，安全序列为 $\langle P_1, P_2, P_1, P_3, P_4 \rangle$ 。